

Parallel Computing - Part 2

Granularity, Map-Reduce & Parallel Mandelbrot Analysis

Week 5 — Thu 12 Mar

Jimmy Jessen Nielsen
jjn@es.aau.dk

Dept. of Electronic Systems
Aalborg University





Parallel Computing - Part 2

Today's Agenda

- ▶ **Guest Lecture:** HPC Resources at AAU — IT Services (60 min) [TBC]
 - ▶ Available compute clusters, storage, and access
 - ▶ Relevant for MP2 Dask cluster (L07) and beyond
- ▶ **Break** (10 min)
- ▶ **Theory:** Granularity, Load Balancing & Map-Filter-Reduce (30 min)
 - ▶ Fine vs. coarse granularity; load balancing strategies
 - ▶ Graham's bound, makespan, and load imbalance factor
 - ▶ Map-filter-reduce pattern and `Pool.map`
- ▶ **Studio:** Chunk size + Map-Filter-Reduce + Chunked Mandelbrot
 - ▶ Quick milestone overview → independent work
 - ▶ Soft wrap-up ~15:30; session runs until 16:15

MP2 is open (Multiprocessing + Dask, due Mon 7 Apr 23:59)

Parallel Computing - Part 2

Pre-Lecture Check



Quick poll - raise your hand:

- ▶ Who listened to the podcast?

Parallel Computing - Part 2

Pre-Lecture Check



Quick poll - raise your hand:

- ▶ Who listened to the podcast?
- ▶ Who watched the video overview?

Parallel Computing - Part 2

Pre-Lecture Check



Quick poll - raise your hand:

- ▶ Who listened to the podcast?
- ▶ Who watched the video overview?
- ▶ Who has parallel Mandelbrot working from L04?

MP2 is open — Multiprocessing + Dask, due Mon 7 Apr 23:59



Parallel Computing - Part 2

Learning Objectives

- ▶ Learning objectives for this lecture:
 - ▶ Understand **granularity** (fine vs. coarse parallelism)
 - ▶ Know **Graham's bound** for task scheduling
 - ▶ Know how to **balance load** effectively
 - ▶ Master **map-filter-reduce** programming pattern
 - ▶ Extend **parallel Mandelbrot** with tunable chunk size
 - ▶ Perform **comprehensive performance analysis**
 - ▶ Begin MP2 deliverables (multiprocessing)

By the end of today:

- ▶ Deeper understanding of parallel overhead
- ▶ Map-reduce programming skills
- ▶ Optimal chunk size found and documented
- ▶ Comprehensive speedup analysis underway

Agenda



HPC Resources at AAU

Granularity Concepts

Studio

Wrap-up



HPC Resources at AAU

HPC Resources at AAU

Guest Lecture — IT Services [TBC]

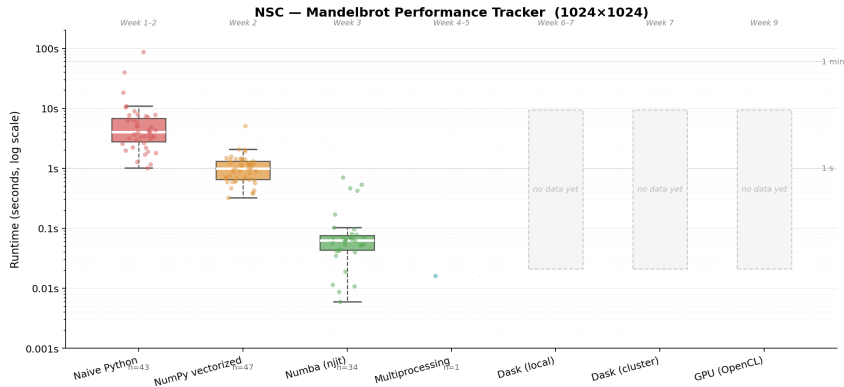


Guest Lecture — IT Services, AAU

Slides provided by speaker.

How Fast Is Your Mandelbrot?

Class Performance Tracker



1024×1024 grid. Boxplot (excl. errors >300 s); dots are individual results.

Updated: March 12, 2026



Granularity Concepts

Granularity

Recap: what we built in L04

In L04 we parallelised Mandelbrot with `Pool.map`:

- ▶ Divided the $N \times N$ grid into `n_workers` equal-sized row chunks — one chunk per worker, each chunk = $N/\text{n_workers}$ rows
- ▶ Each worker received only scalar bounds (no large array over IPC) and computed its own pixel coordinates independently
- ▶ `np.vstack(parts)` reassembled the result in the main process

This is embarrassingly parallel (Eijkhout §2.5.4): each pixel is fully independent, so we can divide the work any way we like.

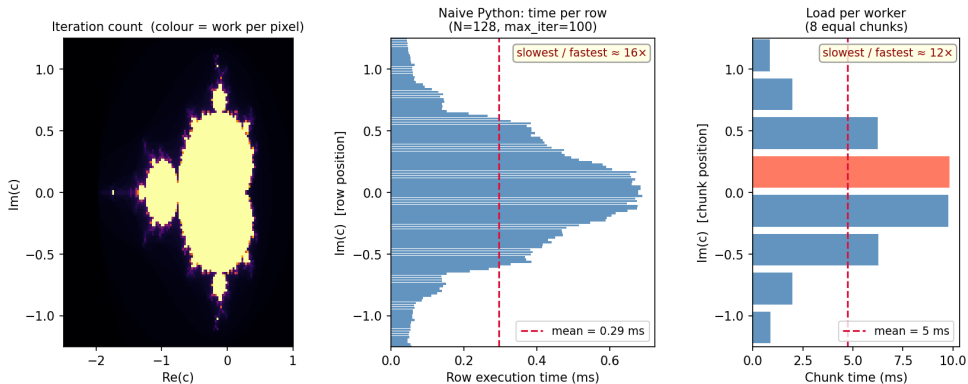
But one question remained:

- ▶ We used exactly `n_workers` chunks — one per process
- ▶ Mandelbrot rows near the set boundary take many more iterations than rows far from it — equal-sized chunks $\neq$ equal work
- ▶ What happens if we use *more* chunks than workers? And how large should those chunks be?

⇒ **Today: granularity theory, then measure it in E1 and MP2 M1–M2.**

Granularity

Load Imbalance: Mandelbrot Evidence



Rows near the set boundary ($\text{Im}(c) \approx 0$) take up to $\sim 16\times$ longer than exterior rows at $\text{max_iter}=100$. Large equal-sized chunks $\Rightarrow$ unequal work.

Granularity

Task Size and Parallel Overhead (*Eijkhout §2.5.6, §2.6.1.4, §2.10.3*)

Solution is overdecomposition: create more tasks than processors so workers pull the next task when free. But how many tasks should we create?

Granularity: *the amount of work a task performs before communicating or synchronising.*

Two competing costs:

- ▶ t_{dispatch} : per-task IPC overhead (grows with task count)
- ▶ $t_{\text{imbalance}}$: idle time from unequal task sizes (shrinks with task count)

n_{chunks}	$= n_{\text{workers}}$	"sweet spot"	$\gg n_{\text{workers}}$
t_{dispatch}	low	moderate	high
$t_{\text{imbalance}}$	high	low	very low

Sweet spot: modest overdecomposition

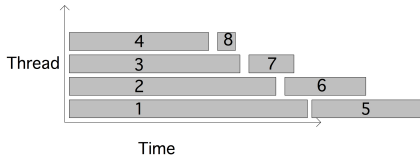
- ▶ Typically $4\text{--}32 \times n_{\text{workers}}$ in practice
- ▶ Each worker handles several chunks in sequence; a slow chunk does not stall all others
- ▶ Beyond the sweet spot, dispatch cost outweighs the gain

Load Balancing

Static vs. Dynamic Scheduling (*Eijkhout §2.10.2*)

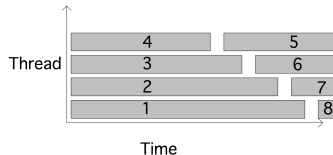
Static scheduling:

- ▶ Work divided and assigned before execution starts
- ▶ Simple, predictable, cache-friendly
- ▶ Suffers when tasks vary in runtime — one slow task stalls all workers waiting for it



Dynamic scheduling:

- ▶ Work assigned on demand as processors become free
- ▶ Absorbs variability automatically
- ▶ Higher overhead per task (coordination cost)
- ▶ `Pool.map()` uses dynamic scheduling by default
- ▶ Needs **many small tasks** to be effective



8 tasks of decreasing runtime on 4 threads: static (left) leaves threads idle; dynamic (right) keeps threads busy.

Scheduling Theory

Graham's Bound (1966)

Makespan T_p : Total wall time from start to last task finishing with p processors.

List scheduling: Assign next available task to the first free processor.

Graham's bound (upper bound on parallel wall time):

$$T_p \leq \underbrace{\frac{1}{p} \sum_{i=1}^m t_i}_{\text{average load}} + \underbrace{\left(1 - \frac{1}{p}\right) t_{\max}}_{\text{longest-task penalty}},$$

where $t_{\max}$ = longest single task, $\sum t_i$ = total work

Key insight: Two competing terms:

- ▶ average load: it is what it is,
- ▶ reduce $t_{\max}$ (use smaller chunks) to tighten the bound.
(*IPC overhead excluded; real T_p is higher.*)

Example — unequal load, $p = 4$ workers:

- ▶ $n_chunks=4$: durations $\approx [0.5, 0.5, 0.5, 3.0]$ s, $t_{\max} = 3.0$ s $\Rightarrow$ bound = $4.5/4 + (3/4) \times 3.0 = 3.4$ s
- ▶ $n_chunks=16$: heavy load split $\Rightarrow t_{\max} \approx 0.75$ s $\Rightarrow$ bound = $4.5/4 + (3/4) \times 0.75 = 1.7$ s $2\times$ better!

Scheduling Theory

Load Imbalance Factor & Mandelbrot

Load Imbalance Factor (LIF):

$$\text{LIF} = \frac{p \cdot T_p}{\sum t_i} - 1 \quad \implies \quad \text{LIF} = \frac{p \cdot T_p}{T_1} - 1$$

- ▶ $\sum t_i$ = total work (sum of all chunk runtimes) = T_1 = your measured serial wall time
- ▶ T_p = parallel wall time on p cores; $p = \text{n_workers}$
- ▶ **LIF = 0**: perfect efficiency — $p \cdot T_p = T_1$, all cores busy the whole time
- ▶ **LIF = 0.5**: 50% overhead — from load imbalance, IPC cost, synchronisation, or any combination
- ▶ Example: $T_1 = 10 \text{ s}$, $p = 8$, $T_p = 1.5 \text{ s} \Rightarrow \text{LIF} = 8 \times 1.5/10 - 1 = 0.2$

What LIF is useful for:

- ▶ **Single diagnostic number** — combines all sources of overhead into one comparable value
- ▶ **Sweep comparisons** — compute LIF for each `n_chunks` value; the minimum locates the sweet spot between dispatch cost and imbalance

Relation to parallel efficiency E from L04: $E = \frac{T_1}{p \cdot T_p} = \frac{1}{1 + \text{LIF}}$

E : “what fraction of the ideal do you achieve?”

LIF: “how much overhead are you paying?”

Scheduling Theory

LIF Worked Example: 8 vs. 32 Chunks

From the Mandelbrot row timing data ($N=128$, $\text{max_iter}=100$, $p=8$ workers):

8 chunks = $1 \times n_workers$ (16 rows each):

- ▶ Mean chunk work: $\frac{T_1}{8} = 0.125 T_1$
- ▶ Heaviest chunk (rows 48–63): $t_{\max} = 0.27 T_1$
- ▶ Imbalance ratio: $t_{\max}/\bar{t} = 2.1 \times$
- ▶ Graham bound:

$$T_p \leq \frac{T_1}{8} + \frac{7}{8} \times 0.27 T_1 = 0.36 T_1$$
- ▶ **LIF = $8 \times 0.36 - 1 = 1.87$** (very high!)
- ▶ Speedup $\approx 1/0.36 = 2.8 \times$ vs ideal $8 \times$

32 chunks = $4 \times n_workers$ (4 rows each):

- ▶ Mean chunk work: $\frac{T_1}{32} = 0.031 T_1$
- ▶ Heaviest chunk (rows 60–63): $t_{\max} = 0.074 T_1$
- ▶ Imbalance ratio: $t_{\max}/\bar{t} = 2.4 \times$
- ▶ Graham bound:

$$T_p \leq \frac{T_1}{8} + \frac{7}{8} \times 0.074 T_1 = 0.19 T_1$$
- ▶ **LIF = $8 \times 0.19 - 1 = 0.52$** (much better)
- ▶ Speedup $\approx 1/0.19 = 5.3 \times$ vs ideal $8 \times$

Note: Graham's bound captures pure load imbalance; actual LIF is higher due to IPC and synchronisation overhead.

Key takeaway: The configurations have comparable $t_{\max}/\bar{t}$ ratio ($\approx 2 \times$), but 32 chunks wins because the *absolute* $t_{\max}$ is $3.6 \times$ smaller — workers idle for much less total time — reflected in LIF.

Map-Filter-Reduce

Functional Programming Pattern

- ▶ A standard pattern for data transformation pipelines
- ▶ Three operations:
 1. **Map:** Apply function to each element independently
 - ▶ Input: [1, 2, 3, 4] Map: $x \rightarrow x^2$ Output: [1, 4, 9, 16]
 2. **Filter:** Keep elements matching a condition
 - ▶ Input: [1, 4, 9, 16] Filter: $x > 5$ Output: [9, 16]
 3. **Reduce:** Combine all elements into a single result
 - ▶ Input: [9, 16] Reduce: sum Output: 25
- ▶ Why it is directly relevant to this course:
 - ▶ **Your Mandelbrot is a map:** apply `compute_pixel` independently to every $(c_{\text{real}}, c_{\text{imag}})$ point in the grid — no element depends on any other
 - ▶ `Pool.map()` works for Mandelbrot *precisely because* the computation is a pure map step (embarrassingly parallel)
 - ▶ The collect step is reduce: `np.vstack(parts)` reassembles the sub-results
 - ▶ Historical notes: a) Google's MapReduce paper (Dean & Ghemawat, 2004) popularised map+reduce for distributed data at scale — inspiring Hadoop, Spark, and Dask.
b) map/filter/reduce themselves originate in functional programming (Lisp, 1950s).

Map-Filter-Reduce

Code example

The same pipeline in three Python styles — all produce the same result:

Setup:

```
from functools import reduce
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
# Pipeline: square each number, keep only values > 30, sum them
```

Style 1: explicit map / filter / reduce (mirrors the theory slide):

```
result = reduce(lambda a, b: a + b,
                filter(lambda x: x > 30,
                      map(lambda x: x**2, data)))
print(result) # 330
```

Style 2: list comprehension (idiomatic Python):

```
result = sum(x**2 for x in data if x**2 > 30)
print(result) # 330
```

Style 3: for-loop (most explicit):

```
result = 0
for x in data:
    sq = x ** 2
    if sq > 30:
        result += sq
print(result) # 330
```

Note: Style 1 maps directly to `pool.map()` — see E2.

Studio

MP2: Multiprocessing + Dask

Due Mon 7 Apr 23:59

New implementations to add (across L04–L07):

- ▶ Multiprocessing — chunked parallel Mandelbrot (L04 + L05 today)
- ▶ Dask local — distributed array on your laptop (L06, Thu 19 Mar)
- ▶ Dask cluster — deploy on AAU Strato (L07, Thu 26 Mar)

What to hand in:

- ▶ Code — .py or .ipynb file(s)
- ▶ Performance notebook — timing table + speedup plots for all implementations
- ▶ Git log — `git log --format="%ad %h %s" --date=short --stat > git_log.txt`
- ▶ Reflection — 150+ words, typed into the Moodle text field (not uploaded)

⇒ **Document your Mandelbrot parallel results from L04 and today — your performance notebook entries count toward MP2.**

Studio Session

Overview — today's plan

Quick walkthrough, then you work independently.

Exercises (practice — no hand-in):

E1 — **Chunk Size** Monte Carlo π : sweep chunk sizes; plot overhead curve (25 min)

E2 — **Map-Filter-Reduce** Build serial pipeline with `map/filter/reduce`, then swap to `Pool.map`; discuss when parallelism helps (25 min)

MP2 Milestones (document results in performance notebook):

M1 — **Chunked Mandelbrot** Add `n_chunks` parameter to `mandelbrot_parallel`; re-run benchmark (20 min)

M2 — **Optimal Chunk Size** Sweep `n_chunks`; find sweet spot for your hardware (15 min)

M3 — **Comprehensive Analysis** Compare all implementations; record speedup table (20 min)

Soft wrap-up ~15:30

- ▶ After 15:30: keep working, ask for help — session runs until 16:15
- ▶ Commit your work before you leave!

Chunk Size Investigation in MC π

E1: Overhead vs. Granularity (25 min)

Goal: Observe how chunk size L affects parallel overhead using Monte Carlo π .

Setup:

- ▶ Total work $N = \text{const}$ (same total compute regardless of L)
- ▶ Vary L from 10 to 1 000 000 samples per chunk
- ▶ Test with $P=1$ (serial) and $P=\text{os.cpu_count()}//2$ (parallel)
- ▶ Print elapsed time for each configuration

What to look for:

- ▶ Serial time: stays roughly flat (computation is constant, no IPC overhead)
- ▶ Parallel time: U-shaped curve — high overhead at small L , poor balance at large L
- ▶ Where is the optimal L for your machine?

✓ **Done?** Identify optimal chunk size → discuss with a neighbour

Chunk Size Investigation

E1: Code example

```
from multiprocessing import Pool
import random, time, os

def monte_carlo_chunk(num_samples):
    """Estimate pi contributions for num_samples random points."""
    inside = 0
    for _ in range(num_samples):
        x, y = random.random(), random.random()
        if x*x + y*y <= 1:
            inside += 1
    return inside

def test_granularity(total_work, chunk_size, n_proc):
    n_chunks = total_work // chunk_size
    tasks = [chunk_size] * n_chunks
    t0 = time.perf_counter()
    if n_proc == 1:
        results = [monte_carlo_chunk(s) for s in tasks]
    else:
        with Pool(processes=n_proc) as pool:
            results = pool.map(monte_carlo_chunk, tasks)
    return time.perf_counter() - t0, 4 * sum(results) / total_work

if __name__ == '__main__':
    total_work = 1_000_000
    n_proc = os.cpu_count() // 2
    chunk_sizes = [10, 100, 1_000, 10_000, 100_000, 1_000_000]
    print(f"{'L':>12} | {'serial (s)':>12} | {'parallel (s)':>12}")
    for L in chunk_sizes:
        t_ser, _ = test_granularity(total_work, L, n_proc=1)
        t_par, pi = test_granularity(total_work, L, n_proc=n_proc)
        print(f"{'L':12d} | {'t_ser':12.4f} | {'t_par':12.4f}   pi={pi:.4f}")
```

Map-Filter-Reduce

E2: Serial pipeline $\rightarrow$ parallel map (25 min)

Data: $N = 1\,000\,000$ random integers in $[10, 100]$

Pipeline: map (subtract 7) $\rightarrow$ filter (keep odd) $\rightarrow$ reduce (sum)

Part 1 — Serial (10 min):

- ▶ Implement the pipeline using `map()`, `filter()`, `reduce()` built-ins
- ▶ Run and verify the result

Part 2 — Parallel (15 min):

- ▶ **Before running — predict:** will `Pool.map()` be faster? Why or why not?
- ▶ Replace `map(subtract_seven, data)` with `pool.map(subtract_seven, data)`
- ▶ Time both versions and compare

After running:

- ▶ Was your prediction correct?
- ▶ How does this compare to the overhead curve you saw in E1?

✓ **Done?** Discuss with a neighbour $\rightarrow$ move on to MP2 milestones

Map-Filter-Reduce

E2: Code example

Setup — data and worker function (module-level for pickling):

```
import random, time
from functools import reduce
from multiprocessing import Pool

N = 1_000_000
data = [random.randint(10, 100) for _ in range(N)]

def subtract_seven(x):
    return x - 7
```

Part 1 — serial pipeline (map / filter / reduce chained):

```
t0 = time.perf_counter()
result_ser = reduce(lambda a, b: a + b,
                    filter(lambda x: x % 2 == 1,
                            map(subtract_seven, data)))
t_serial = time.perf_counter() - t0
```

Part 2 — replace map() with Pool.map():

```
t0 = time.perf_counter()
with Pool() as pool:
    mapped = pool.map(subtract_seven, data)
result_par = reduce(lambda a, b: a + b,
                    filter(lambda x: x % 2 == 1, mapped))
t_parallel = time.perf_counter() - t0

print(f"Serial:    {t_serial:.4f}s    result={result_ser}")
print(f"Parallel:  {t_parallel:.4f}s    result={result_par}")
print(f"Speedup:    {t_serial / t_parallel:.2f}x")
```

Parallel Mandelbrot — MP2 M1

Chunked Mandelbrot (20 min)

Goal: Decouple the number of chunks from the number of workers.

Why: In L04's `mandelbrot_parallel`, one chunk per worker. Mandelbrot has variable work per row — more, smaller chunks improve load balance.

What to do:

- ▶ Add an `n_chunks` parameter to `mandelbrot_parallel` (default: `n_chunks = n_workers`)
- ▶ Verify output still matches the serial result from L04
- ▶ Re-run the L04 worker-count benchmark with `n_chunks = 4 × n_workers`; note any change

✓ **Done?** Verify result matches serial output → commit

Parallel Mandelbrot — MP2 M1

Code example

From L04 (unchanged — add `cache=True` if not already there):

```
@njit(cache=True)
def mandelbrot_pixel(c_real, c_imag, max_iter): ...
@njit(cache=True)
def mandelbrot_chunk(row_start, row_end, N, x_min, x_max, y_min, y_max, max_iter): ...
def _worker(args): return mandelbrot_chunk(*args) # plain Python, must be module-level
```

Extend your L04 `mandelbrot_parallel` to (include `n_chunks`):

```
def mandelbrot_parallel(N, x_min, x_max, y_min, y_max,
                       max_iter=100, n_workers=4, n_chunks=None):
    if n_chunks is None:
        n_chunks = n_workers
    chunk_size = max(1, N // n_chunks)
    chunks, row = [], 0
    while row < N:
        row_end = min(row + chunk_size, N)
        chunks.append((row, row_end, N, x_min, x_max, y_min, y_max, max_iter))
        row = row_end
    tiny = [(0, 8, 8, x_min, x_max, y_min, y_max, max_iter)]
    with Pool(processes=n_workers) as pool:
        pool.map(_worker, tiny) # warm-up: load JIT cache in workers
        parts = pool.map(_worker, chunks)
    return np.vstack(parts)
```

Parallel Mandelbrot — MP2 M1

cache=True: why it matters with multiprocessing

Without `cache=True`, every worker process re-compiles the Numba function from scratch on startup:

- ▶ macOS and Windows use the **spawn** start method for multiprocessing: each worker is a fresh Python process that imports the module and applies `@njit` anew
- ▶ JIT compilation takes 5–30 s per worker; with 4–8 workers this adds up even though they compile in parallel
- ▶ The compiled machine code is also lost when the `Pool` context exits, so the **next** `Pool` re-compiles again

`@njit(cache=True)` **saves the compiled native code to** `__pycache__`:

- ▶ First run compiles once in the main process — workers load from disk on startup
- ▶ Warm-up inside `Pool` only needs a tiny task (8×8 grid) to trigger the cache load, not a full $N \times N$ computation
- ▶ Subsequent runs (and re-opened Pools) skip compilation entirely

Linux uses **fork** by default, which copies the parent's memory (including compiled code) into workers — `cache=True` is less critical there but still good practice.

Parallel Mandelbrot — MP2 M2

Optimal Chunk Size (15 min)

Fix `n_workers` at your L04 optimum. Sweep `n_chunks`, measure wall time, and compute LIF
 $= p \cdot T_p / T_1 - 1$.

Suggested configurations (`n_workers` = your best):

n_chunks	time (s)	vs. 1×	LIF
1× n_workers	—	baseline	—
2× n_workers	—	—	—
4× n_workers	—	—	—
8× n_workers	—	—	—
16× n_workers	—	—	—

Tips:

- ▶ Use `statistics.median()` of 3+ runs
- ▶ Include warm-up before timing
- ▶ LIF minimum = sweet spot
- ▶ Typical sweet spot: 4–8× `n_workers`

✓ **Done?** Record optimal `n_chunks` and LIF in performance notebook (MP2) → commit

Parallel Mandelbrot — MP2 M3

Comprehensive Analysis (20 min)

Create a full performance comparison at 1024×1024 , `max_iter=100`:

1. Speedup table:

Implementation	Time (s)	Speedup
Naive Python	—	$1\times$
NumPy	—	—
Numba (@njit)	—	—
Parallel (opt.)	—	—

2. Speedup vs. core count:

- ▶ Actual vs. ideal (linear)
- ▶ Back-solve implied $s = \frac{1/S_{p^*} - 1/p^*}{1 - 1/p^*}$
- ▶ Is your s smaller than your L04 result?
Better load balance $\Rightarrow$ lower apparent s

✓ **Done?** Record all results in performance notebook (MP2) $\rightarrow$ commit

3. Brief recommendation:

- ▶ What settings give the best time?
- ▶ Is parallelisation worth it on your hardware?
Compare speedup and LIF to your serial Numba baseline
- ▶ Write 2–3 sentences in your performance notebook

4. Submit to Performance Tracker:

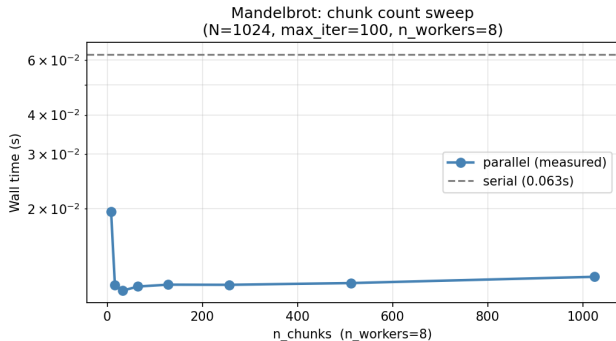
- ▶ Enter your best multiprocessing time on Moodle
(1024×1024 , Implementation: "Multiprocessing")



Wrap-up

Results: Chunk Sweep

M1 Max, $p = 8$ workers, $N = 1024$, $\text{max_iter}=100$



- ▶ **Serial baseline:** $T_1 = 0.063\text{ s}$ ($N = 1024$, multiprocessing, $p = 8$ workers)
- ▶ **$1 \times n_workers$ (8 chunks):** LIF = 1.50 — one boundary chunk stalls the pool; speedup only $3.2 \times$
- ▶ **Sweet spot at $4 \times n_workers$ (32 chunks):** LIF = 0.39, speedup = $5.74 \times$
- ▶ **Overhead flat beyond:** LIF stays 0.39–0.55 from 32 to 1024 chunks (1 row/chunk)

Wrap-up

What We Covered Today

► Granularity theory:

- Fine vs. coarse granularity and their tradeoffs
- Graham's bound: makespan and load imbalance

► E1 — Chunk size investigation:

- Overhead vs. load balance tradeoff observed empirically
- Monte Carlo π as a controllable benchmark

► E2 — Map-filter-reduce:

- Serial pipeline with `map/filter/reduce` built-ins
- Swapping to `Pool.map` doesn't help for trivial ops — overhead $\gg$ compute

► MP2 milestones:

- M1: `mandelbrot_parallel` extended with `n_chunks`
- M2: Optimal chunk count found for your hardware
- M3: Comprehensive speedup table started

Before Next Lecture

Preparation for L06 (Thu 19 Mar)

Continue MP2:

- ▶ Finalise multiprocessing Mandelbrot results from today
- ▶ Document all timings in your performance notebook

Prepare for L06 (Distributed Computing with Dask):

1. Motivational and reading material in Moodle
2. Install Dask: `mamba install dask distributed` (or `conda install dask distributed`)
3. Test: `python -c "import dask; print(dask.__version__)"`

Next lecture topics (L06 — Thu 19 Mar):

- ▶ Dask arrays (like NumPy but distributed)
- ▶ Lazy evaluation and task graphs
- ▶ Convert Mandelbrot to Dask